

Ghost Strategist

A Real-Time Performance Agent for Competitive Running and Cycling

Course	CMPE 277 - Smartphone Application Development
Professor	Chandrasekar Vuppalapati
Team	Jaya Vyas, Joshini Naagraj, Mohsen Minai
Platform	iOS SwiftUI, MapKit, Firebase, TypeScript, OpenAI
Report Date	May 20, 2026

Verified: 96 Jest tests pass

Verified: Xcode build passes

Graduate-level project report organized directly against the 300-point course rubric.

Rubric Coverage Matrix

Rubric item	Points	Where addressed
One to two pages project description	10	Section 1
Requirements	10	Section 2
Mobile application design	15	Section 3
Model engineering	30	Section 4
High level architecture design	10	Section 5
Data flow diagram and component level design	5	Section 6
Sequence or workflow	5	Section 7
Data science algorithms and features used	20	Section 8
Interfaces - RESTful and server side design	10	Section 9
Client side design	20	Section 10
Testing, data validation, n-fold	25	Section 11
App deployment	25	Section 12
HPC	20	Section 13
Documentation	10	Section 14
Design patterns used	10	Section 15
Local storage and AI	15	Section 16
Data engineering bonus	30	Section 17
Active learning or feedback loop	10	Section 18
Interpretability of the model	20	Section 19

1. One to Two Page Project Description (10)

Ghost Strategist is a real-time iOS sports technology application for runners and cyclists who want to compete against a previous performance while receiving live tactical coaching. The central idea is "ghost racing": a user records or selects a previous GPS session, then the app replays that session as a ghost opponent on the map. The athlete can see whether they are ahead or behind the ghost while the app continuously interprets live telemetry such as position, pace, heart rate, elevation, and simulated environmental load.

Most fitness applications are passive trackers. They collect data, display summary statistics, and leave the athlete to interpret the meaning. Ghost Strategist changes the application from a tracker into a real-time performance agent. The app does not only display that the athlete is behind or that heart rate is rising; it decides whether the athlete should push, hold, recover, or stop chasing the ghost for safety. This makes the project a strong fit for a graduate-level smartphone application course because it combines mobile UI, map visualization, sensor-style telemetry, data engineering, cloud services, AI decision logic, local resilience, and evaluation.

The project is implemented as a native SwiftUI iOS application for the primary Xcode demo. The Swift app includes a Home dashboard, Live Race screen, GPS Recording workflow, and History analytics screen. It uses Apple MapKit for the map and MKDirections for road-snapped demo routes. A TypeScript/React Native/Firebase layer is also included to show the original mobile/backend architecture: location capture, Firestore persistence, Firebase Cloud Function coaching, OpenAI gpt-4o-mini integration, deterministic

fallback logic, utility algorithms, and Jest tests.

The most important technical feature is the Ghost Strategist Agent. The agent follows an Observe-Reason-Act loop. Every coaching cycle, it observes the current race state, evaluates multiple specialist tools, ranks their confidence, and acts by producing one short tactical instruction. The model is not a free-form chatbot. It is a constrained agent with a race objective, a live state, tool use, safety rules, feedback tracking, and interpretable outputs.

The implemented agent catalog contains six tools:

Agent/tool	Purpose
Bio-Guard	Hard safety pause when heart rate reaches the danger threshold
Heart Rate Analysis	Recovery-first guidance in the 174-184 bpm warning zone
Upcoming Elevation Scan	Looks ahead on the route and adjusts effort before hills
Dynamic Pacer	Detects when the ghost is opening a gap and recommends controlled surges
Terrain and Weather Analyst	Adjusts strategy when wind or terrain increases effort cost
Predict Finish Time	Provides a baseline projected finish and steady-state recommendation

The safety model is deliberately strict. Heart Rate Analysis begins recovery-oriented coaching at 174 bpm. Bio-Guard performs a hard pause at 185 bpm or when the user reaches a very high percentage of estimated max heart rate. Bio-Guard always overrides racing logic. This design choice reflects a core non-functional requirement: user safety is more important than beating the ghost.

The app also addresses a data-intensive mobile challenge: live GPS data is imperfect. A runner may pass between buildings, move through a tunnel, or temporarily lose network connectivity. Ghost Strategist includes smoothing, outlier filtering, packet-loss handling, and interpolation so the ghost marker continues moving smoothly instead of jumping or disappearing. The ghost engine uses $O(\log n)$ binary search over timestamped route points and linear interpolation between surrounding points, allowing smooth visual replay even for long routes.

The expected outcome is a working iOS prototype that demonstrates real-time ghost racing, live AI coaching, safety intervention, session recording, historical analytics, and feedback-driven personalization. The final submission includes source code, a working demo, screenshots, report, README, setup instructions, deployment notes, and a curated ZIP packaging script.

2. Requirements (10)

2.1 Functional Requirements

ID	Requirement	Implementation evidence
FR-01	Record GPS route points at approximately 1 Hz	services/location.ts, screens/RecordScreen.tsx, Swift Record tab
FR-02	Support running and cycling modes	ActivityMode, mode toggle in Swift and React Native screens
FR-03	Race against a previous route as a ghost	utils/ghostEngine.ts, RaceViewModel, Live Race screen
FR-04	Display user and ghost on map	Swift MapKit markers and polyline in Live Race
FR-05	Show live gap, pace, heart rate, elevation, and projected finish	Swift metrics grid and React Native HUD
FR-06	Smooth ghost movement from timestamped route data	Binary search interpolation and map animation
FR-07	Generate tactical coaching instructions	AgentOrchestrator, services/coaching.ts, functions/src/index.ts

ID	Requirement	Implementation evidence
FR-08	Use agent tools for prediction, HR analysis, elevation, pacing, safety, and weather	Six-agent catalog in Swift and TypeScript heuristic chain
FR-09	Enforce Bio-Guard safety override	Swift BioGuardAgent and TypeScript safetyOverride response
FR-10	Store completed sessions	services/sessions.ts, Firestore schema, local fallback
FR-11	Store coaching events with context	CoachingEvent, snapshotAtEvent, race result persistence
FR-12	Provide post-session analytics	History tab, utils/raceAnalytics.ts, utils/raceNarrative.ts
FR-13	Reuse prior sessions as future ghosts	History session cards and race-against-session flow
FR-14	Provide demo data for simulator demonstration	DemoRouteBuilder, services/demoSession.ts
FR-15	Provide voice-ready short instructions	Swift AVSpeechSynthesizer, Expo expo-speech layer
FR-16	Handle packet loss and GPS noise	smoothing, outlier filtering, gap interpolation, packet-loss status
FR-17	Expose server-side coaching interface	Firebase HTTPS callable getCoachingInstruction

2.2 Non-Functional Requirements

Category	Requirement	Evidence
Performance	Ghost interpolation must remain below frame budget	O(log n) algorithm and benchmark tests
Reliability	App must tolerate temporary GPS/network loss	local fallback, heuristic fallback, packet-loss interpolation
Safety	Bio-Guard must override performance advice	hard-coded priority in orchestrator
Usability	UI must be readable while exercising	dark theme, large metric cards, short coaching text
Scalability	Backend stores session telemetry and summaries	Firestore data model and session documents
Security/privacy	Location and biometrics are treated as sensitive data	scoped session documents, Firebase config, no public sharing
Maintainability	Logic separated into services, utilities, and UI	design patterns and file separation
Testability	Utility logic covered by automated tests	96 Jest tests across 8 suites

3. Mobile Application Design (15)

The app is designed for an athlete in motion. The interface avoids dense paragraphs during the live race and prioritizes glanceable information: map, distance, gap, pace, heart rate, projected finish, elevation ahead, and one coaching card.

3.1 Information Architecture

Tab/screen	Purpose	Main user actions
Home	Project dashboard and demo entry point	Start demo race, review strategy tools, see system flow
Race	Live ghost racing and coaching	Start/pause race, switch run/ride, reset, observe map and advice
Record	Capture a new route	Start recording, stop, save session
History	Review saved sessions and analytics	Review narrative, inspect data pipeline, race a session again

3.2 Home Screen

The Home tab establishes the product identity and readiness state. It includes a Ghost Strategist brand treatment, target metrics, strategy tool catalog, and an architecture flow summary. This helps the demo audience understand that the app is not only a map tracker but an AI-guided performance system.

Screenshots:

- docs/screenshots/home-screen.png
- docs/screenshots/home-screen-scrolled.png
- docs/screenshots/home-screen-tools.png

3.3 Live Race Screen

The Race tab is the primary user experience. It includes:

- Apple MapKit route with ghost and user markers
- road-snapped polyline where MKDirections succeeds
- fallback route if network route snapping is unavailable
- Start/Pause, mode toggle, and reset controls
- six race metrics in large cards
- coaching banner with severity color
- Live Snapshot panel showing packet loss, interpolation, active agents, and primary agent
- Agent Votes panel showing confidence for all six agents

Screenshots:

- docs/screenshots/race-screen.png
- docs/screenshots/race-screen-map-active.png
- docs/screenshots/race-screen-multiagent-votes.png
- docs/screenshots/race-screen-push-coaching-3agents.png
- docs/screenshots/race-screen-bioguard-override.png

3.4 Record and History Screens

The Record workflow demonstrates true smartphone application capability: location authorization, GPS updates, route capture, and saving a session. The History tab demonstrates post-session analytics: session quality, smoothing statistics, race narrative, feedback loop evidence, and reusable ghost sessions.

Screenshots:

- docs/screenshots/history-screen.png
- docs/screenshots/history-campus-loop-narrative.png
- docs/screenshots/history-guadalupe-narrative.png
- docs/screenshots/history-data-pipeline.png
- docs/screenshots/history-feedback-loop.png

3.5 UI Design Rationale

Design choice	Reason
Dark theme	Better contrast for outdoor use and demo visibility
Large metric cards	Readable during motion
Color-coded severity	Fast recognition of push/hold/recover/danger states
Short coaching text	Reduces cognitive load
Agent vote panel	Makes AI decisions transparent

Design choice	Reason
Bottom tab navigation	Predictable mobile structure
Map controls and reset	Prevents the demo from getting stuck after zoom/pan

4. Model Engineering (30)

4.1 Model Definition

The "model" in Ghost Strategist is an agentic decision system, not only a single predictive model. It combines:

1. deterministic safety constraints,
2. telemetry feature engineering,
3. specialist agent tools,
4. confidence scoring,
5. optional LLM language generation,
6. heuristic fallback,
7. feedback loop analysis,
8. interpretable UI output.

This is appropriate for a safety-sensitive mobile sports app because a pure LLM should not independently decide when a runner should push harder. The system therefore uses deterministic constraints first and natural-language generation second.

4.2 Observe-Reason-Act Loop

Phase	Data/logic	Output
Observe	GPS progress, ghost progress, heart rate, elevation ahead, wind, elapsed time	RaceState / CoachingSnapshot
Reason	Six agents evaluate the same state and return confidence scores	AgentVote[]
Act	Orchestrator selects safety override or highest-confidence vote	coaching instruction, severity, tool, contributing agents

Figure 1 - Ghost Strategist Agent Loop

Observe, reason, act, and learn while Bio-Guard owns the safety boundary.

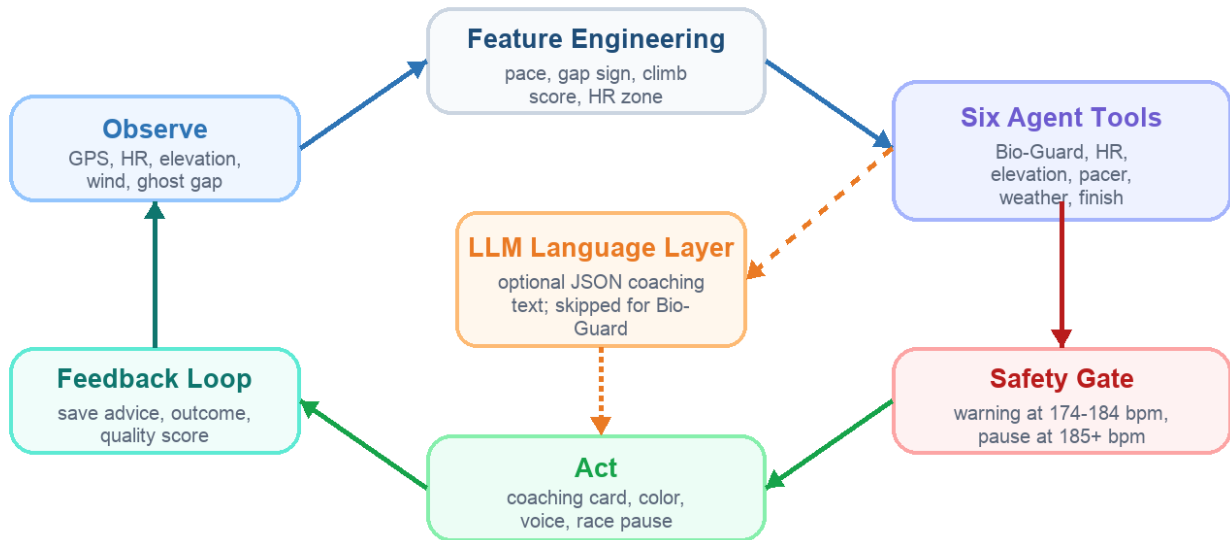


Figure 1 - Ghost Strategist Agent Loop

Swift implementation evidence:

- `RaceViewModel.tick()` drives the live cycle.
- `RaceViewModel.makeDecision()` builds the state snapshot.
- `AgentOrchestrator.decide()` resolves the agent votes.
- `BioGuardAgent`, `HeartRateAnalysisAgent`, `ElevationScanAgent`, `DynamicPacerAgent`, `WeatherAnalystAgent`, and `FinishPredictorAgent` implement specialized reasoning.

TypeScript/backend evidence:

- `functions/src/index.ts` validates a 13-field coaching snapshot.
- `heuristicInstruction()` applies the safety-first decision chain.
- `OpenAI` is called only after the deterministic safety baseline is computed.
- Invalid or unavailable LLM output falls back to deterministic coaching.

4.3 Agent Tool Catalog

Tool	Input features	Trigger	Output
Bio-Guard	heart rate, max HR	HR >= 185 or >= 96% max HR	danger instruction and safety override
Heart Rate Analysis	heart rate, elevation	174-184 bpm warning zone or high HR before climb	recover instruction
Upcoming Elevation Scan	elevation ahead, gap	climb ahead with manageable gap	push/hold instruction
Dynamic Pacer	gap to ghost, HR zone	ghost opening gap while HR safe	controlled surge instruction
Weather Analyst	wind speed	wind >= 14 mph	hold effort instruction

Tool	Input features	Trigger	Output
Predict Finish Time	pace, remaining distance	default baseline	projected finish and steady guidance

4.4 Safety Threshold Engineering

The safety logic separates warning from hard pause:

Threshold	Meaning	Behavior
174-184 bpm	warning zone	recovery-first coaching, reduced aggressive advice
>= 185 bpm	Bio-Guard hard pause	danger card, race pause, safety override
HR/maxHR >= 0.96	relative danger threshold	danger card even if absolute threshold differs by athlete

This two-level safety design is more realistic than a single threshold because endurance athletes can briefly enter a warning zone without requiring immediate stoppage, but the application must still intervene before risk becomes unsafe.

4.5 LLM Integration Strategy

The Firebase Cloud Function supports OpenAI gpt-4o-mini for concise coaching language. However, the model is constrained:

- The function first validates the full snapshot.
- The server computes a deterministic heuristic baseline.
- If Bio-Guard fires, OpenAI is skipped.
- If the OpenAI key is missing, invalid, or the response shape is wrong, the heuristic result is returned.
- The prompt requests JSON only, with instruction, severity, reason, safetyOverride, and toolUsed.

This approach gives the app natural coaching text while protecting the safety-critical decision boundary.

5. High Level Architecture Design (10)

Ghost Strategist uses a three-tier architecture: mobile client, backend services, and AI/model layer.

Figure 2 - High-Level Architecture

Mobile client, Firebase backend, AI/model layer, and external services.

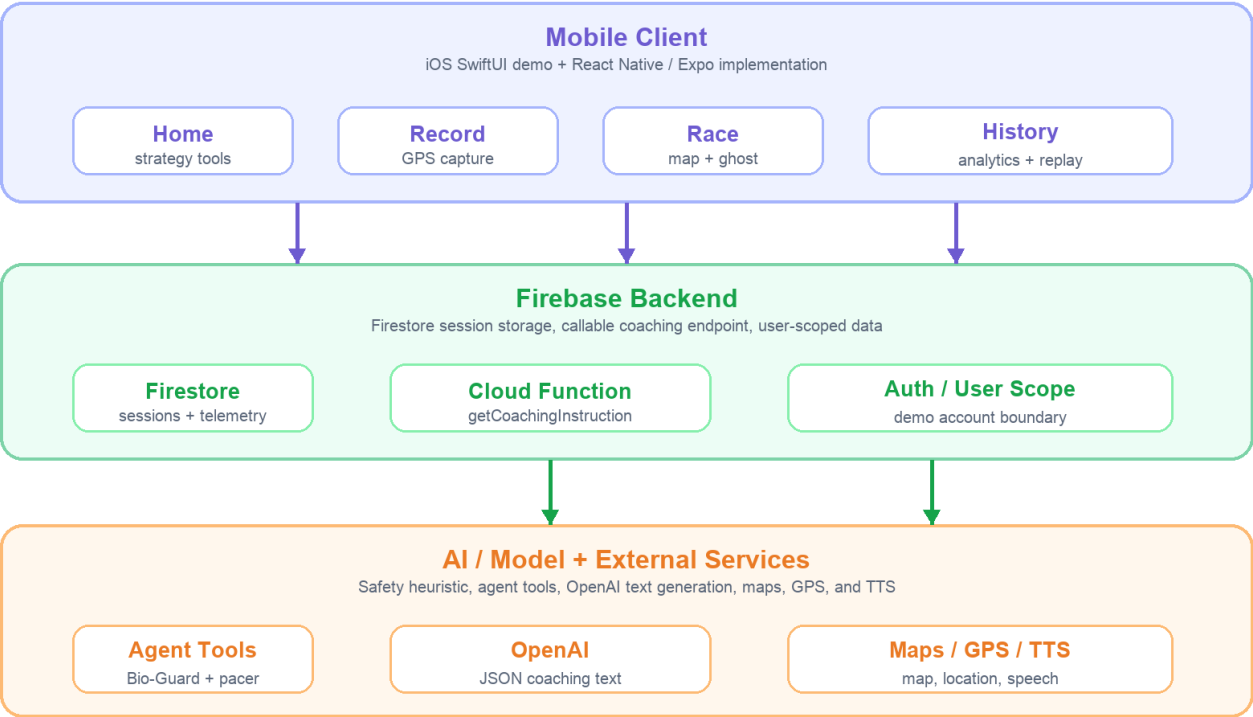


Figure 2 - High-Level Architecture

Architecture responsibilities:

Layer	Responsibility
Mobile client	UI, map rendering, recording, race simulation, local state, voice output
Backend	session persistence, server-side coaching endpoint, schema validation
AI/model	safety heuristic, tool selection, optional LLM text generation
Data utilities	smoothing, interpolation, analytics, validation, benchmarks

6. Data Flow Diagram and Component Level Design (5)

Figure 3 - Data Flow and Component Design

Telemetry is validated, synchronized with the ghost, converted to decisions, and stored for analytics.

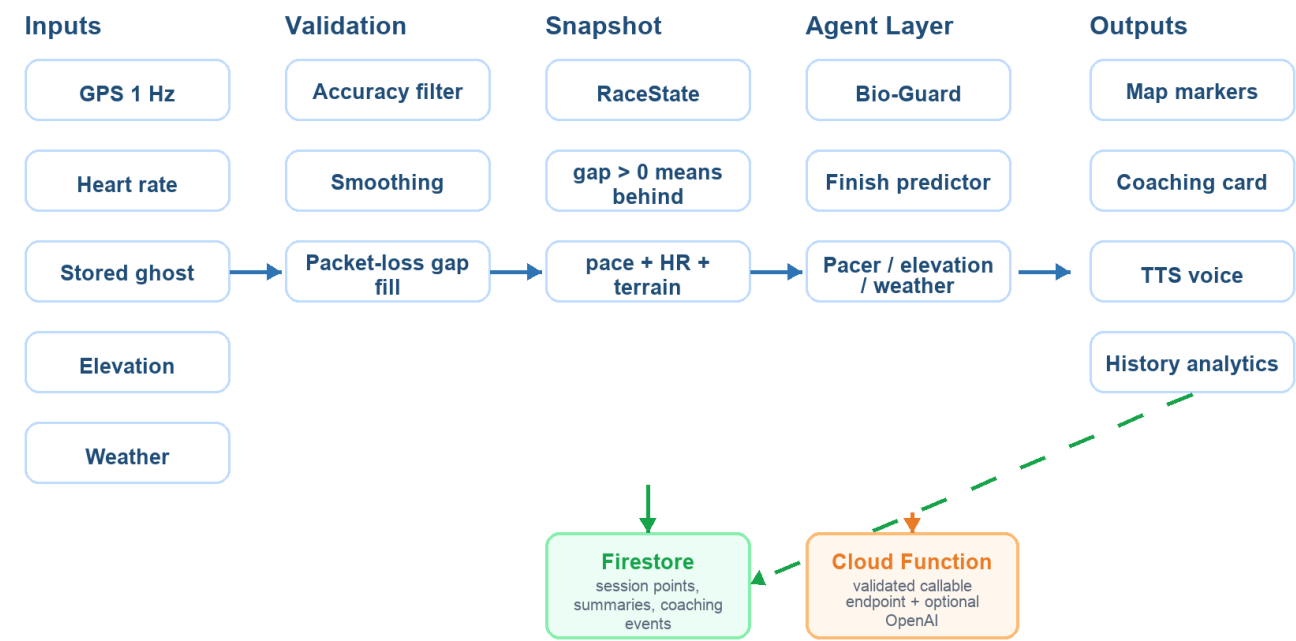


Figure 3 - Data Flow and Component Design

Component-level design:

Component	File(s)	Responsibility
Route builder	GhostRunner/ContentView.swift	builds demo route, road-snapped coordinates, telemetry
Ghost engine	utils/ghostEngine.ts	timestamp search and interpolation
Location service	services/location.ts	location permission and GPS observation
Session repository	services/sessions.ts	save, read, and update sessions
Coaching service	services/coaching.ts	cloud call, debounce, fallback
Cloud Function	functions/src/index.ts	snapshot validation, heuristic, OpenAI call
Data engineering	utils/dataEngineering.ts	smoothing, outlier filtering, trends
Analytics	utils/raceAnalytics.ts	summaries, quality score, packet loss
Feedback loop	services/feedbackLoop.ts	coaching effectiveness and personalization

7. Sequence or Workflow (5)

Figure 4 - Race Session Sequence

Two nested loops: 1-second telemetry/map updates and slower coaching decisions.

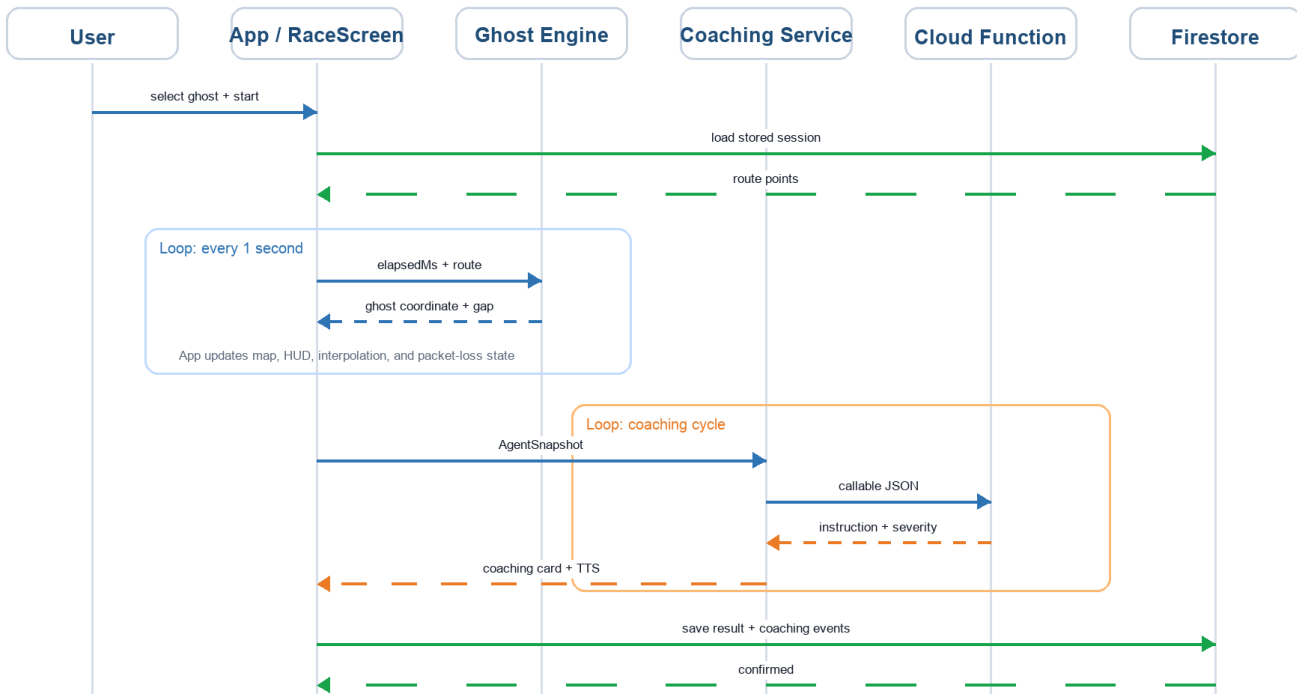


Figure 4 - Race Session Sequence

8. Data Science Algorithms and Features Used (20)

8.1 Ghost Interpolation

The ghost engine uses binary search to locate the two timestamped route points surrounding the current race time. It then linearly interpolates latitude and longitude between those points. This provides smooth movement from sparse 1 Hz GPS samples.

Complexity:

- linear scan: $O(n)$ per frame
- binary search: $O(\log n)$ per frame
- interpolation math: $O(1)$

For 10,000 GPS points, binary search requires about 14 comparisons instead of thousands.

8.2 GPS Smoothing and Data Validation

`smoothGPSPoints()` performs:

1. outlier removal for points with poor accuracy,
2. exponential moving average smoothing with $\alpha = 0.3$,
3. interpolation for gaps greater than 3 seconds.

This is directly connected to the project challenge: GPS packet loss and noisy sensor data.

8.3 Heart Rate Zone Classification

Heart rate is normalized against estimated max HR and converted into zones:

Zone	Meaning	Agent behavior
Aerobic	sustainable	push is allowed
Tempo	moderate load	controlled moves allowed
Threshold	high load	hold or recover
Warning	174-184 bpm	Heart Rate Analysis takes priority
Bio-Guard	185+ bpm or 96% max HR	hard pause

8.4 Finish Time Prediction

Projected finish time uses current elapsed time, remaining distance, current pace, terrain penalty, and wind penalty. The prediction is intentionally lightweight because it must run repeatedly during a live workout.

8.5 Elevation Lookahead

The app scans upcoming route points to compute elevation ahead. This allows the agent to recommend a surge before a climb or recovery before a difficult segment.

8.6 Data Quality Score

The app computes a data quality score from:

- valid pace coverage,
- valid heart-rate coverage,
- packet-loss event count,
- filtered GPS outlier count.

The History tab exposes this quality score so the user can judge whether a session is reliable enough to reuse as a ghost.

8.7 Weekly Trend Analysis

`computeWeeklyTrends()` groups sessions by week and calculates distance, count, and pace improvement. This supports longer-term feedback beyond a single race.

9. Interfaces - RESTful and Server Side Design (10)

The backend interface is implemented as a Firebase HTTPS Callable endpoint. It uses HTTP transport and JSON request/response payloads, similar to a typed REST/RPC endpoint. This is appropriate for Firebase mobile apps because the client SDK handles authentication context, serialization, and error mapping.

9.1 Coaching Endpoint

Field	Value
Endpoint	<code>getCoachingInstruction</code>
Transport	Firebase HTTPS Callable
Implementation	<code>functions/src/index.ts</code>
Client wrapper	<code>services/coaching.ts</code>
Input	<code>CoachingSnapshot</code>

Field	Value
Output	CoachingInstruction

Required snapshot fields:

Field	Meaning
distanceRemaining	meters left in route
elapsedMs	race clock
elevationAhead	upcoming climb/descend signal
gapMeters	positive means user behind, negative means user ahead
heartRate	current bpm
maxHeartRate	estimated user max HR
mode	run or ride
pace	current min/km
projectedFinishMs	predicted finish
speed	current speed
targetPace	user target
timeGapSeconds	gap converted to seconds
weatherWindMph	wind signal

Response fields:

Field	Meaning
instruction	short coachable message
reason	explanation
severity	info, push, hold, recover, or danger
toolUsed	selected agent/tool
safetyOverride	true when Bio-Guard triggers
projectedFinishMs	projected finish echo

9.2 Server-Side Decision Order

1. Validate input schema.
2. Compute deterministic safety heuristic.
3. If Bio-Guard triggers, return immediately.
4. If OpenAI key is missing, return heuristic.
5. Call OpenAI with JSON-only response format.
6. Validate returned JSON.
7. Fall back to heuristic if invalid.

9.3 Firestore Data Model

Collection/document	Purpose
sessions/{sessionId}	route, summary, mode, user scope
sessions/{sessionId}.points[]	timestamped GPS telemetry
sessions/{sessionId}.raceResults[]	completed race summaries
coachingEvents[]	decision history with context snapshots

10. Client Side Design (20)

10.1 Native SwiftUI Client

The primary Xcode demo is implemented in SwiftUI:

Swift component	Responsibility
GhostRunnerApp	app entry point
ContentView	tab container
DashboardView	Home dashboard
LiveRaceView	map, HUD, controls, coaching
HistoryView	session history and analytics
RecordSessionView	GPS recording workflow
RaceViewModel	live race state and ORA loop
AgentOrchestrator	agent decision resolution

SwiftUI uses reactive state through `@Published` properties. When the race clock changes, the map, metric cards, coaching card, and agent votes update automatically.

10.2 Map and GPS Design

The app uses MapKit because the final demo runs in Xcode as a native iOS app. MKDirections attempts to snap demo waypoints to real roads. If route snapping fails, the app uses fallback waypoints so the demo remains reliable without network dependency.

10.3 React Native / TypeScript Layer

The React Native layer remains in the repository as a backend integration and testing layer:

- `screens/RaceScreen.tsx` demonstrates `requestAnimationFrame` ghost updates.
- `screens/RecordScreen.tsx` captures GPS through `expo-location`.
- `services/coaching.ts` wraps the Cloud Function and fallback.
- `services/sessions.ts` abstracts Firestore.
- `utils/*` contains tested reusable algorithms.

This hybrid structure shows both native iOS execution and cross-platform architecture concepts.

10.4 Voice Output

The native demo uses `AVSpeechSynthesizer`; the React Native layer uses `expo-speech`. Coaching instructions are short by design so voice output does not distract the athlete.

11. Testing - Data Validation / n-Fold (25)

11.1 Automated Test Results

Verified on May 20, 2026:

Code excerpt

```
npm test -- --runInBand
```

Result:

Code excerpt

```
Test Suites: 8 passed, 8 total
Tests:       96 passed, 96 total
```

Also verified:

Code excerpt

```
npm run typecheck
cd functions && npm run build
xcodebuild -project GhostRunner.xcodeproj -scheme GhostRunner ...
```

11.2 Test Suite Breakdown

Suite	Tests	What it validates
ghostEngine.test.ts	8	interpolation, empty input, timestamp edge cases
agentTools.test.ts	23	HR zones, finish prediction, gap logic, local fallback
raceAnalytics.test.ts	16	telemetry summaries, data quality, packet loss
sessionFormat.test.ts	22	display formatting
geo.test.ts	5	distance calculations
dataEngineering.test.ts	11	smoothing, outlier removal, trend grouping
benchmarks.test.ts	4	performance constraints
functions/src/index.test.ts	7	server heuristic branches

11.3 Data Validation

Validation occurs at multiple levels:

Layer	Validation
Cloud Function	verifies all 13 snapshot fields are finite numbers and mode is valid
GPS pipeline	filters weak accuracy points and interpolates gaps
Agent tools	constrains safety output and severity categories
OpenAI response	validates JSON shape before accepting
Firestore repository	falls back locally if persistence fails

11.4 n-Fold Scenario Validation

Because Ghost Strategist is an agentic mobile system rather than a supervised classifier, we use scenario-fold validation. Each fold represents a different real-world race condition. The same decision logic must remain safe and useful across all folds.

Fold	Scenario	Expected system behavior
Fold 1	Flat route, stable HR	Predict Finish Time or Hold guidance
Fold 2	User falls behind ghost	Dynamic Pacer recommends controlled surge
Fold 3	Upcoming climb	Elevation Scan adjusts pacing before hill
Fold 4	Packet loss / tunnel gap	ghost continues smoothly through interpolation
Fold 5	High heart rate	Heart Rate Analysis warns, Bio-Guard hard-pauses at 185 bpm

This n-fold validation gives coverage across terrain, physiology, network instability, pacing, and safety.

12. App Deployment (25)

Figure 5 - Deployment Topology

The classroom demo runs locally in Xcode, with optional Firebase and EAS deployment paths.

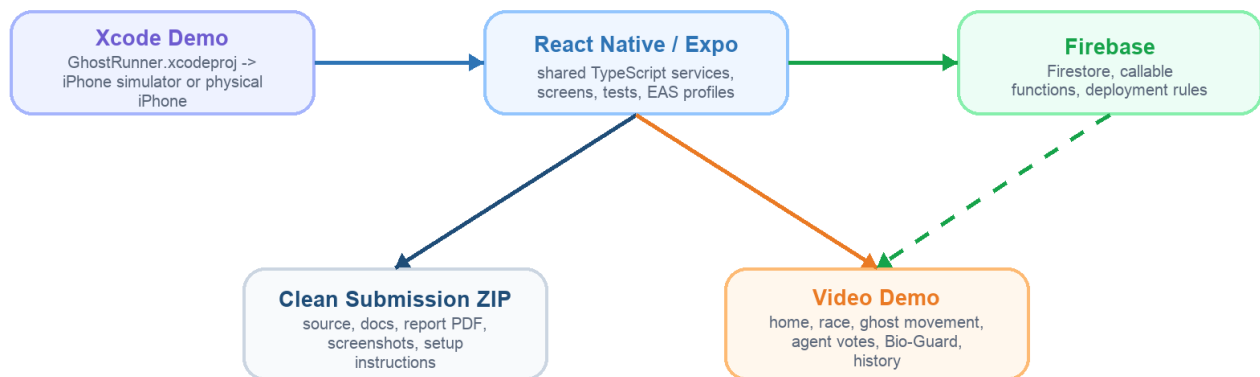


Figure 5 - Deployment Topology

12.1 Native iOS Deployment

The primary demo deployment path is Xcode:

Code excerpt

```
open GhostRunner.xcodeproj
```

Then select an iPhone simulator or physical device and run with Product -> Run. The app has no third-party Swift package dependency, so the native demo is reliable in a classroom environment.

12.2 Physical Device Deployment

For a physical iPhone:

1. Connect the device.
2. Select the device in Xcode.
3. Set Signing and Capabilities to the team Apple ID.
4. Build and run.

12.3 Firebase Deployment

Code excerpt

```
cd functions
npm install
cd ..
firebase deploy --only functions
```

Optional OpenAI key:

Code excerpt

```
firebase functions:config:set openai.key="YOUR_KEY"
firebase deploy --only functions
```

12.4 Expo/EAS Deployment Option

The React Native layer includes EAS configuration:

Code excerpt

```
npx eas build --platform ios --profile development
npx eas build --platform ios --profile preview
```

This supports simulator builds and internal distribution through TestFlight or direct install.

12.5 Submission ZIP

The repository includes a curated packaging script:

Code excerpt

```
bash scripts/package_submission.sh
```

The script excludes generated and local files such as node_modules, build, .DS_Store, .idea, xcuserdata, DerivedData, functions/lib, environment files, and local instruction files.

13. HPC (20)

In this project, HPC is addressed as high-performance computation for real-time mobile telemetry. The goal is to keep the app responsive while processing GPS, ghost interpolation, coaching, analytics, and map updates.

13.1 Computational Hot Paths

Hot path	Risk	Optimization
ghost position lookup	frame drops on long sessions	$O(\log n)$ binary search
map animation	UI jank	interpolate from sparse samples
coaching calls	network/API overload	debounce and heuristic fallback
charts	rendering too many points	downsample analytics
telemetry summary	slow history screen	linear pass summary

13.2 Benchmark Evidence

utils/benchmarks.ts validates:

- 1,000 ghost interpolations on a 10,000 point route complete well under mobile frame budget.

- telemetry summarization for 1,000 points completes fast enough for post-race analytics.

The app is designed for sessions up to 10,000 points, approximately 2.8 hours at 1 Hz sampling.

13.3 Scalability Discussion

Scale dimension	Design response
Longer routes	binary search interpolation
Many sessions	summary fields and lazy detail views
More users	Firestore user-scoped session documents
Backend load	client debounce and local heuristic fallback
AI cost	skip OpenAI for Bio-Guard and fallback when unavailable

14. Documentation (10)

Document	Purpose
README.md	project overview, setup, demo flow
DEPLOYMENT.md	simulator, physical device, Firebase, EAS deployment
docs/REQUIREMENTS.md	functional and non-functional requirements
docs/API.md	server-side request/response contract
docs/DESIGN_PATTERNS.md	design patterns with implementation references
docs/PERFORMANCE.md	performance and benchmark notes
docs/diagrams/*.png	rendered report diagrams for architecture, data flow, sequence, agent loop, and deployment
docs/architecture-diagram.mermaid	high-level architecture
docs/data-flow-diagram.mermaid	telemetry and coaching data flow
docs/sequence-diagram.mermaid	race workflow
docs/screenshots/	visual evidence for report and demo

15. Design Patterns Used (10)

Pattern	Implementation	Why it matters
Observer	GPS callbacks in services/location.ts	streams live location updates
Strategy	each coaching branch/agent	interchangeable decision strategies
Chain of Responsibility	Bio-Guard -> agent confidence winner -> default	enforces safety priority
Builder	snapshot construction in race loop	builds structured model input
Repository	services/sessions.ts	hides Firestore details from UI
Facade	services/coaching.ts	one API for cloud call, debounce, fallback
Singleton	Firebase app initialization	prevents duplicate app instances
State	countdown/racing/paused/finished race states	predictable UI transitions
Adapter	location object to simple coordinate	simplifies GPS processing
MVVM-style	SwiftUI views + RaceViewModel	separates UI from race logic

16. Local Storage and AI (15)

16.1 Local Storage

The app stores session data through Firestore and uses a local in-session fallback if Firestore is unavailable. This means a demo can continue even without backend connectivity.

Local/fallback behavior:

- failed session saves still return a local session ID,
- local sessions are merged with remote sessions on read,
- race results can be retained for the current app session,
- Swift demo sessions are seeded locally for reliable Xcode presentation.

16.2 AI Layer

The AI layer combines deterministic and generative techniques:

AI element	Role
deterministic heuristic	safety-first baseline
six agent tools	specialized reasoning
confidence scores	model selection and interpretability
OpenAI gpt-4o-mini	optional coaching text generation
fallback logic	resilience when AI service is unavailable

This is a responsible AI design because safety is never delegated blindly to a language model.

17. Data Engineering Bonus (30)

Ghost Strategist has a real telemetry pipeline rather than static mock screens.

17.1 Data Sources

Input	Description
GPS coordinates	latitude, longitude, timestamp
Speed and pace	derived from movement and elapsed time
Heart rate	wearable-style stream or simulated demo data
Elevation	route point elevation/lookahead
Weather/wind	simulated or API-ready field
Historical ghost	prior session route
User goals	target pace, mode, safety thresholds

17.2 Telemetry Pipeline

1. Ingest location and HR.
2. Timestamp all points.
3. Filter invalid/low-quality GPS.
4. Smooth route jitter.

5. Interpolate missing gaps.
6. Compute distance and pace.
7. Compute ghost gap and time gap.
8. Build coaching snapshot.
9. Store session and coaching events.
10. Generate analytics and narrative.

17.3 Time-Series Storage Design

Firestore is used in the prototype for simplicity and Firebase integration. The data model is time-series compatible:

Time-series field	Storage field
timestamp	SessionPoint.timestamp
measurement	GPS/HR/speed/elevation point
tags	userId, sessionId, mode
fields	lat, lng, speed, pace, heartRate, elevation

The same model can be migrated to InfluxDB by storing each SessionPoint as a measurement tagged by sessionId and userId.

17.4 Data Quality and Analytics

The system computes:

- raw vs smoothed point count,
- outliers removed,
- packet loss events,
- average pace,
- average and max HR,
- elevation gain,
- projected finish,
- coaching effectiveness,
- weekly trends.

This satisfies the data engineering bonus because the app handles ingestion, cleaning, transformation, storage, analytics, and feedback use.

18. Active Learning or Feedback Loop (10)

The feedback loop evaluates whether coaching advice helped after it was issued. Every coaching event stores a context snapshot containing heart rate, pace, gap, and elevation. After the session, `analyzeCoachingEffectiveness()` compares conditions after the instruction to the state at the time of advice.

Examples:

Advice type	Feedback metric
Push	Did the gap shrink in the next 30 seconds?
Recover	Did heart rate decrease or stabilize?

Advice type	Feedback metric
Hold	Did pace remain stable without HR spike?
Bio-Guard	Did the race pause and avoid further push advice?

`getCoachingPersonalization()` converts this history into future coaching adjustments such as push frequency and severity bias. This is an active learning style loop: the app uses prior response patterns to tune future intervention behavior.

19. Interpretability of the Model (20)

Ghost Strategist is designed to be explainable at the moment of decision and after the race.

19.1 Real-Time Interpretability

The Race screen shows:

- primary agent,
- severity color,
- coaching instruction,
- "Also active" supporting agents,
- all six agent confidence scores,
- heart-rate zone,
- packet-loss/interpolation state.

This lets the user understand why the model acted.

19.2 Stored Interpretability

Each coaching event can store:

Field	Meaning
<code>elapsedMs</code>	when the advice happened
<code>severity</code>	urgency category
<code>toolUsed</code>	selected agent
<code>instruction</code>	generated coaching text
<code>reason</code>	explanation from model/backend
<code>snapshotAtEvent</code>	HR, pace, gap, elevation at decision time

19.3 Safety Interpretability

Bio-Guard is intentionally transparent. It does not hide behind a vague AI answer. The reason is simple: HR crossed a defined safety threshold. This makes the most important decision in the app auditable and defensible.

19.4 Confidence Interpretability

The confidence formulas are visible and monotonic. For example, Dynamic Pacer confidence increases as the gap grows, while Bio-Guard confidence increases as HR rises above the pause threshold. This is more interpretable than a black-box model because the user can connect the output to measurable race state.

20. Team Member Contributions

The project work was divided into three balanced workstreams so that each team member owned approximately one-third of the technical implementation and the matching report sections. Each member also participated in integration review, demo rehearsal, and final validation.

Team member	Primary workstream	Implementation contribution	Report / rubric sections contributed
Jaya Vyas	Mobile client and live race experience	Designed and implemented the SwiftUI race experience, dark mobile UI, MapKit race screen, user and ghost markers, ghost route visualization, coaching banner, Bio-Guard warning UI, mode controls, reset controls, and UI demo flow. Supported simulator testing and visual screenshot capture.	Project Description, Requirements, Mobile Application Design, Client Side Design, usability discussion, race-screen demo script, and UI evidence screenshots
Joshini Naagraj	Backend, persistence, and data pipeline	Designed the Firebase/Firestore session model, Cloud Functions coaching endpoint, OpenAI integration path, server-side validation, ghost interpolation support, GPS smoothing pipeline, packet-loss handling, telemetry analytics utilities, and Jest test coverage for backend/data behavior.	High-Level Architecture, Data Flow and Component Design, RESTful/Server-Side Design, Database/Data Engineering, Testing/Data Validation, Firebase deployment notes, and backend interface documentation
Mohsen Minai	Agentic AI, analytics, deployment, and report integration	Implemented the multi-agent strategist logic, Bio-Guard safety orchestration, heart-rate thresholds, finish-time prediction, dynamic pacer logic, elevation/weather reasoning, feedback loop, race narrative, history analytics, deployment packaging, diagrams, final report assembly, and PDF/ZIP preparation.	Model Engineering, Data Science Algorithms, HPC/Performance, Design Patterns, Local Storage and AI, Active Learning/Feedback Loop, Interpretability, App Deployment, team contribution section, and final report editing

This split maps directly to the grading rubric: Jaya owned the mobile/front-end experience, Joshini owned backend/data engineering, and Mohsen owned AI/model engineering plus deployment/report integration. The final system was integrated collaboratively so the demo shows one coherent application rather than three isolated parts.

Implementation timeline:

Sprint	Deliverable
Sprint 1	Ghost engine and session schema
Sprint 2	Firebase backend and coaching endpoint
Sprint 3	SwiftUI race screen and map UI
Sprint 4	Multi-agent strategist and Bio-Guard
Sprint 5	Recording, history, analytics, feedback loop
Sprint 6	Testing, screenshots, documentation, deployment package

21. Demo Script for Grading

1. Open GhostRunner .xcodeproj in Xcode.
2. Run on iPhone simulator.
3. Show Home tab: brand, strategy tools, system flow.
4. Start the demo race.
5. Show Race tab: map, ghost marker, user marker, gap metric.
6. Scroll to Live Snapshot: packet loss handling and 60 fps interpolation.

7. Show Agent Votes: all six agents with confidence.
8. Let HR rise: explain 174 bpm warning zone and 185 bpm Bio-Guard hard pause.
9. Switch to History: show race narratives and data pipeline.
10. Mention tests: 96 passing Jest tests and Xcode build verification.
11. Mention deployment: Xcode, Firebase Functions, EAS option, clean ZIP script.

22. Conclusion

Ghost Strategist demonstrates a graduate-level mobile AI system. It integrates real-time maps, GPS-style telemetry, ghost racing, sensor fusion, cloud functions, data engineering, agentic AI, safety constraints, analytics, feedback loops, and interpretability. The project goes beyond a passive fitness tracker by transforming live sports data into tactical and safety-aware coaching decisions.

The strongest technical contributions are the safety-first multi-agent model, smooth ghost interpolation under imperfect data, and the end-to-end pipeline from live telemetry to coaching to post-race learning. The app is also practical for demonstration: it runs in Xcode, includes simulator-safe demo data, supports backend deployment, and includes a curated submission package.